

Introduction to Generative AI

End Semester Complete Notes

20 Lectures · Full Topic Coverage

Syllabus Overview

- Lectures 1–5: Machine Learning Foundations
- Lectures 6–8: Neural Networks
- Lectures 9–10: NLP and LLM Architecture
- Lectures 11–12: GenAI Modalities and APIs
- Lectures 13–14: Prompt Engineering and Fine-Tuning
- Lectures 15–16: Chatbots and RAG
- Lectures 17–20: Tooling, Agents and Automation

These notes cover the complete syllabus in depth for examination preparation.

Contents

1	Lecture 1: Introduction to Machine Learning	6
1.1	ML vs Traditional Programming	6
1.2	Difference Between AI, ML, and Deep Learning	7
1.3	Types of Learning	7
1.3.1	Supervised Learning	7
1.3.2	Unsupervised Learning	8
1.3.3	Semi-Supervised Learning	8
1.3.4	Reinforcement Learning	8
1.4	Types of ML Tasks	8
1.4.1	Classification	8
1.4.2	Regression	9
1.4.3	Clustering	9
2	Lecture 2: Data Preprocessing	10
2.1	Handling Missing Values	10
2.1.1	Strategies	10
2.2	Feature Scaling	11
2.2.1	Min-Max Normalisation (Rescaling)	11
2.2.2	Standardisation (Z-score Normalisation)	11
2.2.3	Robust Scaling	11
2.3	Data Encoding Methods	12
2.3.1	Label Encoding	12
2.3.2	One-Hot Encoding (OHE)	12
2.3.3	Ordinal Encoding	12
2.3.4	Target Encoding (Mean Encoding)	12
2.3.5	Binary Encoding	12
3	Lecture 3: Supervised Learning — Linear Regression	13
3.1	Linear Regression Assumptions	13
3.2	Linear Regression Model Formulation	13
3.2.1	Simple Linear Regression	13
3.2.2	Multiple Linear Regression	13
3.2.3	Cost Function: Mean Squared Error	14
3.2.4	Ordinary Least Squares (OLS) – Analytical Solution	14
3.2.5	Gradient Descent	14
3.2.6	Regularisation	14
3.3	Limitations of Linear Regression	14

4	Lecture 4: Supervised Learning — Logistic Regression	16
4.1	Logistic Regression Model Formulation	16
4.1.1	From Linear to Probabilistic Output	16
4.1.2	Log-Odds Interpretation	16
4.1.3	Cost Function: Binary Cross-Entropy	16
4.2	Sigmoid Function	17
4.3	Logistic Regression Assumptions	17
4.4	Limitations of Logistic Regression	17
5	Lecture 5: Machine Learning Model Evaluation	19
5.1	Classification Metrics	19
5.1.1	Confusion Matrix	19
5.1.2	Precision and Recall	19
5.1.3	F1 Score	20
5.2	Regression Metrics	20
5.2.1	Mean Absolute Error (MAE)	20
5.2.2	Mean Squared Error (MSE)	20
5.2.3	Root Mean Squared Error (RMSE)	20
5.2.4	R-Squared (R^2)	21
5.2.5	Adjusted R-Squared	21
5.3	Bias and Variance in ML	21
5.4	Bias-Variance Tradeoff	21
5.5	Cross-Validation	22
5.5.1	Hold-Out Validation	22
5.5.2	k -Fold Cross-Validation	22
5.5.3	Leave-One-Out Cross-Validation (LOOCV)	22
5.5.4	Stratified k -Fold	22
6	Lecture 6: Decision Trees	23
6.1	Introduction to Decision Trees	23
6.2	Impurity Measures: Information Gain	23
6.2.1	Entropy	23
6.2.2	Information Gain	24
6.2.3	Gini Impurity	24
6.3	Advantages vs Disadvantages of Decision Trees	24
6.4	Regression Trees	25
7	Lecture 7: Introduction to Neural Networks	26
7.1	Single Cell Perceptron	26
7.2	Single Layer Perceptron	26
7.3	Backpropagation	27
7.3.1	Algorithm Steps	27
7.3.2	Chain Rule	27
8	Lecture 8: Types of Neural Networks	28
8.1	Multi-Layer Perceptron (MLP)	28
8.2	Activation Functions	28
8.3	Recurrent Neural Network (RNN)	29

9	Lecture 9: Natural Language Processing (NLP)	30
9.1	Text Preprocessing	30
9.2	Tokenisation	30
9.2.1	Word Tokenisation	30
9.2.2	Subword Tokenisation (BPE)	31
9.3	Word Embeddings	31
9.3.1	One-Hot Encoding (Bag of Words)	31
9.3.2	Word2Vec	31
9.3.3	GloVe (Global Vectors)	31
9.3.4	FastText	31
9.3.5	Contextual Embeddings	32
9.4	Sequential Modelling	32
9.5	Language Foundations	32
10	Lecture 10: LLM Architecture	33
10.1	Transformers	33
10.1.1	Architecture Overview	33
10.1.2	Positional Encoding	33
10.1.3	Feed-Forward Sub-layer	34
10.2	Self-Attention Mechanism	34
10.2.1	Scaled Dot-Product Attention	34
10.2.2	Multi-Head Attention	34
10.2.3	Causal (Masked) Self-Attention	34
10.3	Ethical Considerations	35
10.3.1	Responsible AI: Bias Mitigation	35
10.3.2	Misinformation Prevention	35
10.3.3	Fairness Principles	35
11	Lecture 11: Generative AI Modalities	36
11.1	Text Models	36
11.1.1	Text Generation	36
11.1.2	Text Summarisation	36
11.2	Image Models	37
11.3	Code Models	37
11.4	Audio Models	37
11.5	Multimodal Models	37
11.6	Ethics in Generative AI	38
12	Lecture 12: Gen AI Commercial APIs	39
12.1	OpenAI API	39
12.2	Google AI API	39
12.3	API Key Management	40
12.4	Costs, Billing and Rate Limits	40
12.5	Word Embeddings via API	41
13	Lecture 13: Prompt Engineering	42
13.1	Prompt Design Principles	42
13.2	Prompt Types	42
13.2.1	Zero-Shot Prompting	42

13.2.2	One-Shot Prompting	42
13.2.3	Few-Shot Prompting	43
13.2.4	Chain-of-Thought (CoT) Prompting	43
13.2.5	Role Prompting	43
13.3	Evaluating LLM Responses	44
13.4	Prompt Refinement Strategies	44
14	Lecture 14: Fine-Tuning	45
14.1	Fine-Tuning vs Prompting	45
14.2	Situations for Fine-Tuning	45
14.3	Instruction-Tuning	46
14.4	Knowledge Sourcing	46
15	Lecture 15: Managing State in Chatbots	47
15.1	Memory Challenges in Chatbots	47
15.2	Multi-Turn Conversation Strategies	47
15.2.1	Full History	47
15.2.2	Windowed History	47
15.2.3	Summary Memory	47
15.3	Context Management	48
15.4	Passing Conversation History	48
16	Lecture 16: Retrieval-Augmented Generation (RAG)	49
16.1	RAG Overview and Beyond Model Knowledge	49
16.2	Grounding LLMs with External Data Integration	49
16.3	Vector Databases	50
16.3.1	Chroma	50
16.3.2	FAISS (Facebook AI Similarity Search)	50
16.4	Embedding Models	51
17	Lecture 17: Rapid Prototyping Tools	52
17.1	GenAI Rapid Prototyping	52
17.2	HTML/CSS Limitations	52
17.3	Introduction to Streamlit	52
17.4	Gradio Introduction	53
17.5	Building ML UIs with Python	54
18	Lecture 18: Agentic AI — Components	55
18.1	Key Components	55
18.1.1	LLM as Brain	55
18.1.2	Tools Integration	55
18.1.3	Memory Management	56
18.2	Planning Mechanisms in Agentic AI	56
18.3	Closed Source LLMs	56
18.3.1	GPT (OpenAI)	56
18.3.2	Gemini (Google)	57
18.4	Open Source LLMs	57
18.4.1	Ollama	57
18.4.2	LLaMA (Meta)	57

18.4.3 Gemma (Google)	57
19 Lecture 19: Agentic AI — Control Flow	58
19.1 ReAct Framework: Reason + Act	58
19.2 State Management of AI Agents	59
19.3 Complex Control Flow	59
19.4 LangGraph	59
19.4.1 Core Concepts	59
20 Lecture 20: Low-Code Automation	61
20.1 n8n Introduction and Visual Workflows	61
20.2 Nodes in n8n	61
20.3 Managing Credentials in n8n	62
20.4 Triggers in Agentic AI Systems	62
20.5 Replicating LangChain Logic in n8n	63
A Quick Reference: Key Formulae	64
B Glossary	65

Chapter 1

Lecture 1: Introduction to Machine Learning

1.1 ML vs Traditional Programming

In traditional programming, a human programmer explicitly codes **rules** that transform **input data** into **output answers**.

$$\text{Data} + \text{Rules} \xrightarrow{\text{Programmer}} \text{Answers}$$

In machine learning, an algorithm **learns rules** from pairs of input data and desired answers. No explicit rule coding is required.

$$\text{Data} + \text{Answers} \xrightarrow{\text{ML Algorithm}} \text{Rules (Model)}$$

Aspect	Traditional Programming	Machine Learning
Rule Origin	Manually written by humans	Learned automatically from data
Adaptability	Fixed; must be reprogrammed	Updates with new data
Best Use Case	Well-defined logic (e.g., calculators)	Pattern-heavy tasks (e.g., image recognition)
Data Requirement	Minimal	Large datasets needed
Interpretability	Fully transparent	Often a “black box”

Table 1.1: Traditional Programming vs Machine Learning

The fundamental shift in ML is from *programming rules* to *learning from data*. This enables systems to handle tasks where rules are too complex or unknown.

1.2 Difference Between AI, ML, and Deep Learning

These three concepts are nested subsets of one another:

$$\mathbf{AI} \supset \mathbf{Machine\ Learning} \supset \mathbf{Deep\ Learning}$$

The broadest field; any technique that enables machines to **mimic human intelligence**. Includes rule-based systems, search algorithms, expert systems, and ML.

A subset of AI where systems **learn from data** to improve performance on tasks without being explicitly programmed. Requires feature engineering.

A subset of ML using **multi-layered artificial neural networks** (ANNs) to automatically learn hierarchical feature representations from raw data.

Property	AI	ML	Deep Learning
Data Needed	Varies	Moderate	Very Large
Feature Engineering	Manual	Manual	Automatic
Computation	Low	Medium	High (GPUs)
Example	Chess engine	Spam filter	Image recognition

Table 1.2: Comparison of AI, ML, and Deep Learning

1.3 Types of Learning

1.3.1 Supervised Learning

Learning from a **labelled dataset** $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ where each input x_i has a known target label y_i . The model learns a mapping $f : \mathcal{X} \rightarrow \mathcal{Y}$.

Goal: Minimise a loss function $\mathcal{L}(\hat{y}, y)$ over training data.

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i)$$

Examples: Email spam detection, house price prediction, medical diagnosis.

1.3.2 Unsupervised Learning

Learning from **unlabelled data** $\mathcal{D} = \{x_i\}_{i=1}^N$. The model discovers hidden patterns or structures without explicit guidance.

Examples: Customer segmentation (clustering), dimensionality reduction (PCA), generative modelling.

1.3.3 Semi-Supervised Learning

Uses a **small amount of labelled data** combined with a **large amount of unlabelled data**. Bridges supervised and unsupervised learning.

$$\mathcal{D} = \underbrace{\{(x_i, y_i)\}_{i=1}^M}_{\text{labelled}} \cup \underbrace{\{x_j\}_{j=M+1}^N}_{\text{unlabelled}}, \quad M \ll N$$

Use case: Web content classification where only a fraction of pages are manually labelled.

1.3.4 Reinforcement Learning

An **agent** learns to take **actions** in an **environment** to maximise cumulative **reward**. No labelled data; the agent learns through trial and error.

Key components:

- **State** $s \in \mathcal{S}$: current situation of the agent
- **Action** $a \in \mathcal{A}$: what the agent can do
- **Reward** $r \in \mathbb{R}$: feedback signal
- **Policy** $\pi(a|s)$: strategy mapping states to actions

Examples: Game playing (AlphaGo), robotic control, RLHF for LLMs.

1.4 Types of ML Tasks

1.4.1 Classification

Predicts a **discrete class label** $y \in \{c_1, c_2, \dots, c_K\}$ for a given input x .

- **Binary:** $K = 2$, e.g., spam/not-spam

- **Multi-class:** $K > 2$, e.g., digit recognition (0–9)
- **Multi-label:** each input can belong to multiple classes

1.4.2 Regression

Predicts a **continuous numerical value** $y \in \mathbb{R}$ for a given input x .

Examples: House price prediction, stock price forecasting, weather temperature.

1.4.3 Clustering

Groups similar data points into **clusters** without predefined labels. An unsupervised task.

Examples: Customer segmentation, document grouping, anomaly detection.

Chapter 2

Lecture 2: Data Preprocessing

Data preprocessing is the critical step of transforming raw data into a clean, consistent format suitable for ML algorithms. “Garbage in, garbage out” – model quality depends directly on data quality.

2.1 Handling Missing Values

Missing data arises from measurement errors, data corruption, or incomplete records. There are three mechanisms:

- **MCAR** (Missing Completely At Random): missingness has no relationship to data
- **MAR** (Missing At Random): missingness depends on observed variables
- **MNAR** (Missing Not At Random): missingness depends on the missing value itself

2.1.1 Strategies

Strategy	Description	When to Use
Deletion	Remove rows/columns with missing values	Low missingness ($< 5\%$)
Mean Imputation	Replace with feature mean $\bar{x}$	Numerical, symmetric data
Median Imputation	Replace with feature median	Numerical, skewed data
Mode Imputation	Replace with most frequent value	Categorical data
KNN Imputation	Use k nearest neighbours	Complex dependencies
Model-based	Train a predictor for the missing feature	High accuracy needed
Forward Fill	Carry last observed value forward	Time series data

Table 2.1: Missing Value Handling Strategies

For numerical features: use **median imputation** when data is skewed (robust to outliers). For categorical features: use **mode imputation**. For time series: use **forward fill**.

2.2 Feature Scaling

Many ML algorithms are sensitive to the **scale** of input features. Features with large ranges dominate those with small ranges, biasing the model.

2.2.1 Min-Max Normalisation (Rescaling)

Scales features to a fixed range, typically $[0, 1]$:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Properties:

- Output always in $[0, 1]$
- Sensitive to outliers (outliers compress the rest)
- Suitable for: neural networks, image pixel values

2.2.2 Standardisation (Z-score Normalisation)

Transforms feature to have zero mean and unit variance:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

where $\mu = \mathbb{E}[x]$ is the mean and $\sigma = \sqrt{\text{Var}(x)}$ is the standard deviation.

Properties:

- Output is approximately $\mathcal{N}(0, 1)$
- Robust to outliers (compared to Min-Max)
- Suitable for: SVM, linear regression, logistic regression, PCA

2.2.3 Robust Scaling

Uses the **interquartile range (IQR)** – highly robust to outliers:

$$x_{\text{scaled}} = \frac{x - Q_1}{Q_3 - Q_1} = \frac{x - \text{median}(x)}{IQR}$$

Method	Formula	Best For
Min-Max	$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$	Bounded range needed
Z-score	$\frac{x - \mu}{\sigma}$	Normally distributed features
Robust	$\frac{x - Q_1}{Q_3 - Q_1}$	Data with outliers

Table 2.2: Feature Scaling Methods

2.3 Data Encoding Methods

ML algorithms require **numerical** inputs, so categorical variables must be encoded.

2.3.1 Label Encoding

Assigns an integer to each category:

$$\{\text{"red"}, \text{"green"}, \text{"blue"}\} \rightarrow \{0, 1, 2\}$$

Label encoding implies an **ordinal relationship** ($0 < 1 < 2$). Suitable for ordinal categories (e.g., “low”, “medium”, “high”) but can mislead algorithms for nominal categories.

2.3.2 One-Hot Encoding (OHE)

Creates a **binary column** for each category:

$$\text{"red"} \rightarrow [1, 0, 0], \quad \text{"green"} \rightarrow [0, 1, 0], \quad \text{"blue"} \rightarrow [0, 0, 1]$$

- No implied ordering
- Problem: **Dummy variable trap** – perfect multicollinearity. Drop one column (use $K - 1$ columns for K categories)
- High cardinality $\Rightarrow$ very sparse, high-dimensional feature space

2.3.3 Ordinal Encoding

Similar to label encoding but **respects a user-defined order**:

$$\{\text{"low"}, \text{"medium"}, \text{"high"}\} \rightarrow \{1, 2, 3\}$$

2.3.4 Target Encoding (Mean Encoding)

Replaces a category with the **mean of the target variable** for that category. Risk: data leakage – must use cross-validated target encoding.

2.3.5 Binary Encoding

Converts category index to binary representation. More efficient than OHE for high-cardinality features.

Rule of thumb: Use OHE for nominal categories with low cardinality (< 10 – 15 categories); use target encoding or embedding for high-cardinality nominal categories; use ordinal encoding for ordered categories.

Chapter 3

Lecture 3: Supervised Learning — Linear Regression

3.1 Linear Regression Assumptions

Linear regression relies on six key assumptions (LINE + two more):

1. **Linearity:** The relationship between X and Y is linear: $\mathbb{E}[Y|X] = X\beta$.
2. **Independence:** Observations are independent of each other.
3. **Homoscedasticity:** Constant variance of residuals: $\text{Var}(\varepsilon_i) = \sigma^2$ for all i .
4. **Normality of Errors:** Residuals $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ (for inference; not strictly required for prediction).
5. **No Multicollinearity:** Independent variables are not highly correlated with each other.
6. **No Autocorrelation:** $\text{Cov}(\varepsilon_i, \varepsilon_j) = 0$ for $i \neq j$ (important in time series).

3.2 Linear Regression Model Formulation

3.2.1 Simple Linear Regression

For one feature x and target y :

$$\hat{y} = \theta_0 + \theta_1 x$$

where θ_0 is the intercept and θ_1 is the slope.

3.2.2 Multiple Linear Regression

For p features $\mathbf{x} = [x_1, x_2, \dots, x_p]^\top$:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_p x_p = \boldsymbol{\theta}^\top \mathbf{x}$$

In matrix form with design matrix $\mathbf{X} \in \mathbb{R}^{N \times (p+1)}$ (first column all 1s for bias):

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$$

3.2.3 Cost Function: Mean Squared Error

$$J(\boldsymbol{\theta}) = \frac{1}{2N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 = \frac{1}{2N} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2$$

3.2.4 Ordinary Least Squares (OLS) – Analytical Solution

Setting $\nabla_{\boldsymbol{\theta}} J = 0$:

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

The normal equation requires $\mathbf{X}^\top \mathbf{X}$ to be invertible (non-singular). This fails when features are perfectly collinear or $N < p$. Time complexity: $\mathcal{O}(p^3)$ – expensive for large p .

3.2.5 Gradient Descent

Iteratively update parameters in the direction of steepest descent:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^{(t)})$$

where $\alpha > 0$ is the learning rate and:

$$\nabla_{\boldsymbol{\theta}} J = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

3.2.6 Regularisation

Ridge Regression (L2):

$$J_{\text{ridge}}(\boldsymbol{\theta}) = \frac{1}{2N} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2$$

Shrinks all coefficients towards zero (no exact zeros). Handles multicollinearity.

Lasso Regression (L1):

$$J_{\text{lasso}}(\boldsymbol{\theta}) = \frac{1}{2N} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_1$$

Produces **sparse** solutions (exact zeros) – performs feature selection.

Elastic Net: Combination of L1 and L2 penalties.

3.3 Limitations of Linear Regression

1. **Assumes linearity:** Cannot capture non-linear relationships without feature engineering.
2. **Sensitive to outliers:** MSE loss squares errors, amplifying outlier influence.

3. **Multicollinearity:** Highly correlated features make coefficient estimates unstable.
4. **Feature scaling required:** Gradient descent converges poorly without scaling.
5. **Not suitable for classification:** Predicted values are unbounded $(-\infty, \infty)$.
6. **Assumes homoscedasticity:** Violating this leads to inefficient estimates.

Chapter 4

Lecture 4: Supervised Learning — Logistic Regression

4.1 Logistic Regression Model Formulation

Despite its name, Logistic Regression is a **classification** algorithm. It models the probability that an input $\mathbf{x}$ belongs to class 1.

4.1.1 From Linear to Probabilistic Output

The linear combination $z = \boldsymbol{\theta}^\top \mathbf{x}$ can range over $(-\infty, +\infty)$, but probabilities must lie in $[0, 1]$. The **sigmoid function** maps $\mathbb{R} \rightarrow (0, 1)$:

$$\hat{p} = P(Y = 1|\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}}}$$

Decision rule:

$$\hat{y} = \begin{cases} 1 & \text{if } \hat{p} \geq 0.5 \\ 0 & \text{if } \hat{p} < 0.5 \end{cases}$$

Note: $\hat{p} \geq 0.5 \iff z \geq 0 \iff \boldsymbol{\theta}^\top \mathbf{x} \geq 0$.

4.1.2 Log-Odds Interpretation

$$\log \frac{\hat{p}}{1 - \hat{p}} = \boldsymbol{\theta}^\top \mathbf{x}$$

The log-odds (logit) is a **linear** function of $\mathbf{x}$.

4.1.3 Cost Function: Binary Cross-Entropy

The maximum likelihood estimate (MLE) leads to the **log loss**:

$$J(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

This is a convex function – gradient descent finds the global minimum.

Gradient:

$$\nabla_{\theta} J = \frac{1}{N} \mathbf{X}^{\top} (\hat{\mathbf{p}} - \mathbf{y})$$

4.2 Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Key properties:

- Range: $(0, 1)$
- $\sigma(0) = 0.5$
- $\sigma(-z) = 1 - \sigma(z)$ (symmetry)
- Derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ (used in backpropagation)
- Saturates for large $|z| \Rightarrow$ **vanishing gradient** problem

4.3 Logistic Regression Assumptions

1. **Binary outcome:** Dependent variable is binary (0/1). (Multinomial LR extends to multi-class.)
2. **Independence:** Observations are independent.
3. **Linearity of log-odds:** $\log(p/(1 - p))$ is linearly related to predictors.
4. **No multicollinearity:** Features should not be highly correlated.
5. **Large sample size:** Requires large N for reliable estimation (rule of thumb: 10 events per predictor).
6. **No outliers:** Extreme values can heavily influence the decision boundary.

4.4 Limitations of Logistic Regression

1. **Linearly separable data only:** Creates a linear decision boundary; fails on complex, non-linear patterns.
2. **Feature engineering required:** Must manually create polynomial/interaction features for non-linear problems.
3. **Multicollinearity:** Unstable coefficients when predictors are correlated.
4. **Sensitive to outliers:** Can shift the decision boundary significantly.
5. **Not probabilistically calibrated:** Predicted probabilities may not be well-calibrated without adjustment.

6. **Binary by default:** Requires one-vs-rest (OvR) or softmax extension for multi-class.

Chapter 5

Lecture 5: Machine Learning Model Evaluation

5.1 Classification Metrics

5.1.1 Confusion Matrix

A confusion matrix summarises the performance of a classification model by comparing actual vs predicted labels:

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Table 5.1: Confusion Matrix (Binary Classification)

- **TP** (True Positive): Correctly predicted positive
- **TN** (True Negative): Correctly predicted negative
- **FP** (False Positive): Incorrectly predicted positive (Type I Error)
- **FN** (False Negative): Incorrectly predicted negative (Type II Error)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy is **misleading** for imbalanced datasets. If 95% of data is class 0, predicting all class 0 gives 95% accuracy but is useless.

5.1.2 Precision and Recall

$$\text{Precision} = \frac{TP}{TP + FP} \quad (\text{of all predicted positives, how many are actually positive?})$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad (\text{of all actual positives, how many did we catch?})$$

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (\text{of all actual negatives, how many did we correctly reject?})$$

Precision-Recall Tradeoff: Increasing the classification threshold $\uparrow$ precision but $\downarrow$ recall, and vice versa.

5.1.3 F1 Score

The harmonic mean of Precision and Recall:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

Generalised F_β score: Weights recall β times as important as precision:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$$

Use $\beta > 1$ when recall is more important (e.g., cancer screening); $\beta < 1$ when precision is more important (e.g., spam filtering).

5.2 Regression Metrics

5.2.1 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Properties: Same units as target. Robust to outliers. Always non-negative.

5.2.2 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Properties: Penalises large errors more. Units are squared. Differentiable (useful for optimisation).

5.2.3 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Properties: Same units as target. Most commonly reported. Sensitive to outliers.

5.2.4 R-Squared (R^2)

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

- $R^2 = 1$: perfect prediction
- $R^2 = 0$: model is no better than predicting the mean
- $R^2 < 0$: model is worse than predicting the mean

5.2.5 Adjusted R-Squared

R^2 always increases when adding features, even irrelevant ones. Adjusted R^2 penalises for adding non-informative predictors:

$$\bar{R}^2 = 1 - (1 - R^2) \cdot \frac{N - 1}{N - p - 1}$$

where p is the number of predictors and N is the sample size.

5.3 Bias and Variance in ML

Error from incorrect assumptions in the learning algorithm. High bias leads to **underfitting**: the model is too simple to capture the underlying pattern.

$$\text{Bias} = \mathbb{E}[\hat{f}(x)] - f(x)$$

Error from sensitivity to small fluctuations in training data. High variance leads to **overfitting**: the model memorises training data but fails on new data.

$$\text{Variance} = \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$

5.4 Bias-Variance Tradeoff

The expected prediction error (for a given x) decomposes as:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}^2[\hat{f}(x)]}_{\text{model error}} + \underbrace{\text{Variance}[\hat{f}(x)]}_{\text{model sensitivity}} + \underbrace{\sigma_\epsilon^2}_{\text{irreducible noise}}$$

Scenario	Train Error	Test Error	Diagnosis
High Bias	High	High	Underfitting
High Variance	Low	High	Overfitting
Good Fit	Low	Low	Optimal

Table 5.2: Bias-Variance Tradeoff Scenarios

Solutions:

- Underfitting: increase model complexity, add features, reduce regularisation
- Overfitting: simplify model, add regularisation (L1/L2), get more data, dropout, early stopping

5.5 Cross-Validation

5.5.1 Hold-Out Validation

Split data into train ($\sim 70\text{--}80\%$) and test ($\sim 20\text{--}30\%$) sets. Fast but estimate is noisy.

5.5.2 k -Fold Cross-Validation

1. Split data into k equal **folds**
2. For each fold $i = 1, \dots, k$: train on remaining $k - 1$ folds, evaluate on fold i
3. Average performance across all k iterations

$$\text{CV Score} = \frac{1}{k} \sum_{i=1}^k \text{Score}_i$$

Typical values: $k = 5$ or $k = 10$.

5.5.3 Leave-One-Out Cross-Validation (LOOCV)

Special case of k -fold with $k = N$. Nearly unbiased but computationally expensive.

5.5.4 Stratified k -Fold

Ensures each fold maintains the class distribution of the full dataset. Important for imbalanced datasets.

Use **stratified k -fold cross-validation** ($k = 5$ or $k = 10$) as the standard evaluation strategy. It gives a reliable estimate of generalisation performance while being computationally feasible.

Chapter 6

Lecture 6: Decision Trees

6.1 Introduction to Decision Trees

A tree-structured model where:

- Each **internal node** tests a feature condition (e.g., $x_j \leq t$)
- Each **branch** represents the outcome of the test
- Each **leaf node** holds a class label (classification) or value (regression)

The tree is built top-down using **recursive binary splitting**: at each node, choose the feature and threshold that best separates the data.

Key hyperparameters:

- `max_depth`: maximum depth of the tree (prevents overfitting)
- `min_samples_split`: minimum samples required to split a node
- `min_samples_leaf`: minimum samples required in a leaf

6.2 Impurity Measures: Information Gain

6.2.1 Entropy

Measures the impurity or disorder in a set S with K classes:

$$H(S) = - \sum_{k=1}^K p_k \log_2 p_k$$

where p_k is the proportion of class k in S . Convention: $0 \log 0 = 0$.

- $H = 0$: perfectly pure (all same class)
- $H = 1$: maximally impure (50-50 split for binary)

6.2.2 Information Gain

The reduction in entropy achieved by splitting on feature A :

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

where S_v is the subset of S for which feature A has value v .

The algorithm selects the feature $A^* = \arg \max_A IG(S, A)$.

Problem with Information Gain: Biased towards features with many values. Solved by **Gain Ratio** (C4.5 algorithm):

$$\text{GainRatio}(S, A) = \frac{IG(S, A)}{H_A(S)}, \quad H_A(S) = - \sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

6.2.3 Gini Impurity

Used by CART (Classification and Regression Trees):

$$\text{Gini}(S) = 1 - \sum_{k=1}^K p_k^2$$

- Gini = 0: pure node
- Gini = 0.5: maximally impure (binary case)
- Computationally faster than entropy (no logarithm)

6.3 Advantages vs Disadvantages of Decision Trees

Advantages	Disadvantages
Highly interpretable (white-box)	Prone to overfitting without pruning
Handles both numerical and categorical features	Unstable: small data changes $\Rightarrow$ very different trees
No feature scaling required	Biased towards high-cardinality features (with IG)
Handles missing values well	Greedy splits are locally optimal, not globally
Non-parametric (no distributional assumptions)	Poor at extrapolation (regression)
Fast prediction at inference time	Creates axis-aligned boundaries only

Table 6.1: Decision Trees: Advantages vs Disadvantages

6.4 Regression Trees

Regression trees predict continuous values. Instead of class labels, leaf nodes store the **mean** of training targets in that region.

Splitting criterion: Minimise **MSE** (or equivalently, maximise variance reduction):

$$\text{Best split} = \arg \min_{j, t} [\text{MSE}(S_{\text{left}}) + \text{MSE}(S_{\text{right}})]$$

$$\text{MSE}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2$$

Ensemble methods overcome individual tree limitations: **Random Forests** (bagging many trees) reduce variance. **Gradient Boosting** (sequential trees) reduces bias. Both are among the most powerful ML algorithms in practice.

Chapter 7

Lecture 7: Introduction to Neural Networks

7.1 Single Cell Perceptron

The simplest neural unit, inspired by biological neurons. It computes a weighted sum of inputs, adds a bias, and applies an activation function:

$$\hat{y} = \phi \left(\sum_{j=1}^p w_j x_j + b \right) = \phi \left(\mathbf{w}^\top \mathbf{x} + b \right)$$

where $\mathbf{w}$ are weights, b is the bias, and ϕ is the activation function.

Perceptron learning rule:

$$w_j \leftarrow w_j + \alpha(y_i - \hat{y}_i)x_{ij}$$

The single perceptron can only learn **linearly separable** functions. It cannot learn XOR, which motivated the development of multi-layer networks.

7.2 Single Layer Perceptron

A single layer of multiple perceptrons, one per output class. Equivalent to linear/logistic regression. Can only represent linear decision boundaries.

For a single layer with K output units:

$$\hat{y}_k = \phi \left(\mathbf{w}_k^\top \mathbf{x} + b_k \right), \quad k = 1, \dots, K$$

7.3 Backpropagation

An efficient algorithm to compute gradients of the loss function with respect to all weights in a neural network, using the **chain rule** of calculus.

7.3.1 Algorithm Steps

1. **Forward Pass:** Compute activations layer by layer from input to output; compute loss $\mathcal{L}$.
2. **Backward Pass:** Compute gradient of loss w.r.t. output $\frac{\partial \mathcal{L}}{\partial \hat{y}}$, then propagate gradients backwards through the network using the chain rule.
3. **Update:** Use gradients to update all weights via gradient descent.

7.3.2 Chain Rule

For a composition $f(g(x))$:

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

For layer ℓ with pre-activation $z^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ and activation $\mathbf{a}^{(\ell)} = \phi(z^{(\ell)})$:

$$\delta^{(\ell)} = \frac{\partial \mathcal{L}}{\partial z^{(\ell)}} = \left(\mathbf{W}^{(\ell+1)\top} \delta^{(\ell+1)} \right) \odot \phi'(z^{(\ell)})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \delta^{(\ell)} \mathbf{a}^{(\ell-1)\top}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \delta^{(\ell)}$$

Chapter 8

Lecture 8: Types of Neural Networks

8.1 Multi-Layer Perceptron (MLP)

A feedforward neural network with one or more **hidden layers** between the input and output layers. Also called a fully-connected (dense) network or Deep Neural Network (DNN).

Architecture:

- **Input layer:** d input features
- **Hidden layers:** L layers with h_ℓ units each
- **Output layer:** K outputs (sigmoid for binary, softmax for multi-class)

Forward pass through layer ℓ :

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)})$$

Universal Approximation Theorem: An MLP with a single hidden layer containing sufficiently many neurons can approximate any continuous function to arbitrary precision.

8.2 Activation Functions

Activation functions introduce **non-linearity**, enabling networks to learn complex patterns.

Function	Formula	Range	Use Case
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Binary output
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Hidden layers (RNN)
ReLU	$\max(0, z)$	$[0, \infty)$	Hidden layers (default)
Leaky ReLU	$\max(\alpha z, z), \alpha = 0.01$	$(-\infty, \infty)$	Avoids dying ReLU
ELU	z if $z > 0$ else $\alpha(e^z - 1)$	$(-\alpha, \infty)$	Smoother than ReLU
GELU	$z \Phi(z)$	$\approx (-0.17, \infty)$	Transformers
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$	$(0, 1)$, sums to 1	Multi-class output

Table 8.1: Common Activation Functions

Vanishing Gradient Problem: Sigmoid and Tanh saturate at extremes, producing gradients near zero. Deep networks stop learning. ReLU alleviates this.

Dying ReLU Problem: ReLU neurons can get stuck outputting 0 permanently. Leaky ReLU and ELU provide small gradients for $z < 0$.

8.3 Recurrent Neural Network (RNN)

A neural network with **feedback connections** – the output of a layer at time t feeds back as input at time $t + 1$. Designed for **sequential data** (time series, text, speech).

Hidden state update:

$$\mathbf{h}_t = \phi(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

$$\mathbf{o}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

Types:

- **One-to-many:** single input, sequence output (image captioning)
- **Many-to-one:** sequence input, single output (sentiment analysis)
- **Many-to-many:** sequence input, sequence output (machine translation)

Limitations of vanilla RNN:

- **Vanishing gradient:** Gradients shrink over long sequences
- **Exploding gradient:** Gradients grow uncontrollably (fixed by gradient clipping)
- Cannot capture very long-range dependencies

LSTM (Long Short-Term Memory): Introduces **gates** (input, forget, output) to selectively remember/forget information. Solves vanishing gradient for moderate sequence lengths.

GRU (Gated Recurrent Unit): Simpler gating mechanism than LSTM; similar performance with fewer parameters.

Chapter 9

Lecture 9: Natural Language Processing (NLP)

9.1 Text Preprocessing

Raw text must be cleaned and standardised before modelling:

1. **Lowercasing:** “Hello” → “hello”
2. **Punctuation/Special character removal**
3. **Stop word removal:** Remove high-frequency, low-information words (“the”, “is”, “and”)
4. **Stemming:** Reduce to root form by rule (“running” → “run”, “flies” → “fli”)
5. **Lemmatisation:** Reduce to dictionary base form (“running” → “run”, “better” → “good”)
6. **Contraction expansion:** “don’t” → “do not”
7. **HTML tag removal, URL normalisation**

Stemming is faster but less accurate. **Lemmatisation** is slower (requires POS tagging) but more linguistically correct.

9.2 Tokenisation

The process of splitting text into smaller units called **tokens**. Tokens can be words, subwords, characters, or sentences.

9.2.1 Word Tokenisation

Splits on whitespace/punctuation: “Hello world!” → [“Hello”, “world”, “!”]

Problem: Large, open vocabulary. Unseen words (OOV) at test time.

9.2.2 Subword Tokenisation (BPE)

Byte-Pair Encoding (BPE): Iteratively merges the most frequent pair of adjacent characters/tokens.

- Used by GPT, RoBERTa
- Handles OOV words by decomposing into known subwords
- Vocabulary size is fixed (e.g., 50,000)

WordPiece: Similar to BPE but merges based on maximising language model likelihood. Used by BERT.

SentencePiece: Language-agnostic, works on raw text without pre-tokenisation. Used by T5, XLNet.

9.3 Word Embeddings

A dense, low-dimensional vector representation of a word that captures semantic meaning. Semantically similar words have similar vectors.

9.3.1 One-Hot Encoding (Bag of Words)

Each word is a binary vector of dimension $|V|$ (vocabulary size). Sparse, no semantic information.

9.3.2 Word2Vec

Two architectures:

- **CBOW (Continuous Bag of Words):** Predict centre word from context words
- **Skip-gram:** Predict context words from centre word

Key properties:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

Captures analogy, synonymy, and semantic relationships.

9.3.3 GloVe (Global Vectors)

Trained on co-occurrence statistics of the entire corpus. Combines local and global context.

9.3.4 FastText

Extends Word2Vec by representing words as bags of character n-grams. Better handles rare words and morphologically rich languages.

9.3.5 Contextual Embeddings

Unlike static embeddings, contextual embeddings change based on context:

- **ELMo**: Bidirectional LSTM-based; “bank” in “river bank” vs “bank account” gets different vectors
- **BERT / GPT**: Transformer-based; the current standard

9.4 Sequential Modelling

For NLP tasks where order matters:

- **RNNs/LSTMs/GRUs**: Process sequences left-to-right (and optionally right-to-left for bidirectional)
- **Seq2Seq**: Encoder-decoder architecture for translation, summarisation
- **Attention mechanism**: Allows decoder to focus on relevant parts of the input sequence

9.5 Language Foundations

N-gram Language Model: Probability of next word given $n - 1$ previous words:

$$P(w_t | w_{t-n+1}, \dots, w_{t-1})$$

Perplexity: Standard measure of language model quality. Lower is better:

$$\text{Perplexity}(W) = P(w_1, w_2, \dots, w_N)^{-1/N} = 2^{-\frac{1}{N} \sum_t \log_2 P(w_t | w_{1:t-1})}$$

Tasks in NLP: POS tagging, Named Entity Recognition (NER), parsing, machine translation, sentiment analysis, question answering, summarisation.

Chapter 10

Lecture 10: LLM Architecture

10.1 Transformers

A neural network architecture introduced by Vaswani et al. (2017) in “Attention Is All You Need”. Relies entirely on **self-attention** (no recurrence), enabling massive parallelisation and scaling to billions of parameters.

10.1.1 Architecture Overview

The original Transformer consists of:

- **Encoder:** Processes input sequence into contextual representations. Used in BERT-style models.
- **Decoder:** Generates output sequence autoregressively. Used in GPT-style models.
- **Encoder-Decoder:** Both components for sequence-to-sequence tasks. Used in T5, BART.

Each encoder/decoder block contains:

1. Multi-Head Self-Attention
2. Position-wise Feed-Forward Network (FFN)
3. Layer Normalisation (LayerNorm)
4. Residual connections (skip connections)

10.1.2 Positional Encoding

Since self-attention has no inherent sense of order, position information is injected via **positional encodings** added to token embeddings:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

10.1.3 Feed-Forward Sub-layer

Applied independently to each position:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

Typically, inner dimension $4d_{\text{model}}$.

10.2 Self-Attention Mechanism

Each token attends to all other tokens in the sequence (including itself) to compute a weighted contextual representation.

10.2.1 Scaled Dot-Product Attention

Given input matrix $\mathbf{X}$, three projections are learned:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

Attention output:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

The scaling factor $\frac{1}{\sqrt{d_k}}$ prevents dot products from growing large (which would push softmax into saturation).

10.2.2 Multi-Head Attention

Run h attention operations in parallel with different learned projections:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\mathbf{W}^O$$

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

Each head can attend to different aspects (syntax, coreference, etc.).

10.2.3 Causal (Masked) Self-Attention

In decoder (autoregressive) models (e.g., GPT), a **causal mask** prevents attending to future tokens:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right) \mathbf{V}$$

where $M_{ij} = -\infty$ if $j > i$, else 0.

10.3 Ethical Considerations

10.3.1 Responsible AI: Bias Mitigation

LLMs trained on internet text inherit societal biases:

- **Algorithmic bias:** Model outputs reflect and potentially amplify historical biases in data (gender, race, religion)
- **Mitigation:** Dataset curation, debiasing techniques, adversarial debiasing, fairness-constrained training, RLHF with diverse feedback

10.3.2 Misinformation Prevention

LLMs can generate plausible-sounding but false information (**hallucination**):

- **Hallucination:** Confidently stated falsehoods
- **Solutions:** Retrieval-augmented generation (RAG), citations, output verification, Constitutional AI

10.3.3 Fairness Principles

- **Individual fairness:** Similar individuals should receive similar treatment
- **Group fairness:** Protected groups should have comparable outcomes
- **Transparency:** Users should know they are interacting with AI
- **Privacy:** Training data should respect user consent

Chapter 11

Lecture 11: Generative AI Modalities

11.1 Text Models

11.1.1 Text Generation

A model that assigns probabilities to sequences of tokens and generates text by sampling from the next-token distribution:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t | w_1, \dots, w_{t-1})$$

Decoding strategies:

- **Greedy decoding:** $\hat{w}_t = \arg \max_w P(w | w_{<t})$. Fast but repetitive.
- **Beam search:** Maintains top- k partial sequences. Better quality than greedy.
- **Top- k sampling:** Sample from top k tokens. More diverse.
- **Top- p (nucleus) sampling:** Sample from smallest set of tokens whose cumulative probability $\geq p$. Adaptive and widely used.
- **Temperature scaling:** $P'(w) \propto P(w)^{1/T}$. High $T \rightarrow$ more random; low $T \rightarrow$ more deterministic.

11.1.2 Text Summarisation

- **Extractive:** Select and concatenate important sentences from source
- **Abstractive:** Generate new text that captures key information (requires seq2seq or LLM)

Evaluation: ROUGE (Recall-Oriented Understudy for Gisting Evaluation) metrics.

11.2 Image Models

Generative Adversarial Networks (GANs):

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

Generator G produces fake images; discriminator D distinguishes real from fake.

Variational Autoencoders (VAEs): Encode input to latent space $\mathbf{z} \sim \mathcal{N}(\mu, \sigma^2)$; decode to reconstruct/generate.

Diffusion Models (DALL-E, Stable Diffusion, Midjourney):

- **Forward process:** Gradually add Gaussian noise over T steps
- **Reverse process:** Learn to denoise step-by-step (U-Net with attention)
- **Conditioning:** CLIP text embeddings guide image generation

11.3 Code Models

LLMs trained or fine-tuned on large code corpora:

- **Codex / GitHub Copilot:** GPT-based, trained on GitHub code
- **Code Llama:** Meta's code-specific LLM
- **StarCoder, DeepSeek-Coder:** Open-source alternatives

Tasks: Code completion, code generation from natural language, bug detection, code explanation, test generation.

11.4 Audio Models

- **Text-to-Speech (TTS):** Tacotron, FastSpeech, XTTS
- **Speech-to-Text (STT / ASR):** Whisper (OpenAI), DeepSpeech
- **Music Generation:** MusicLM, AudioCraft
- **Representation:** Raw waveform, Mel spectrogram, codec tokens

11.5 Multimodal Models

Models that process and generate across multiple modalities:

- **GPT-4V, Gemini:** Text + image input
- **CLIP (Contrastive Language-Image Pretraining):** Learns aligned text and image embeddings via contrastive learning
- **Flamingo:** Vision-language few-shot learner
- **LLaVA:** Open-source multimodal LLM

11.6 Ethics in Generative AI

- **Deepfakes:** Photorealistic fake media; identity misuse, disinformation
- **Copyright:** Training on copyrighted data; generated content ownership
- **Content safety:** NSFW generation, harmful use cases
- **Environmental cost:** Training large models requires enormous compute and energy
- **Watermarking:** Technical methods to identify AI-generated content

Chapter 12

Lecture 12: Gen AI Commercial APIs

12.1 OpenAI API

Base URL: <https://api.openai.com/v1>

Key endpoints:

- /chat/completions: Multi-turn chat with GPT models
- /embeddings: Generate vector embeddings
- /images/generations: DALL-E image generation
- /audio/transcriptions: Whisper speech-to-text

Chat Completions example:

```
1 from openai import OpenAI
2 client = OpenAI(api_key="YOUR_API_KEY")
3
4 response = client.chat.completions.create(
5     model="gpt-4o",
6     messages=[
7         {"role": "system", "content": "You are a helpful
8         assistant."},
9         {"role": "user", "content": "What is machine learning?"
10        }
11    ],
12    temperature=0.7,
13    max_tokens=200
14 )
15 print(response.choices[0].message.content)
```

Listing 12.1: OpenAI Chat Completions

12.2 Google AI API

Models: Gemini Ultra, Gemini Pro, Gemini Flash

Key features:

- Very long context window (up to 2M tokens for Gemini 1.5)
- Native multimodal (text, image, audio, video, code)
- Available via **Google AI Studio** and **Vertex AI**

```
1 import google.generativeai as genai
2 genai.configure(api_key="YOUR_API_KEY")
3
4 model = genai.GenerativeModel("gemini-pro")
5 response = model.generate_content("Explain neural networks.")
6 print(response.text)
```

Listing 12.2: Google Gemini API

12.3 API Key Management

- **Never hardcode** API keys in source code
- Store in **environment variables**: `os.environ.get("OPENAI_API_KEY")`
- Use `.env` files with `python-dotenv`; add to `.gitignore`
- Use secrets managers in production (AWS Secrets Manager, HashiCorp Vault)
- **Rotate keys** regularly; revoke compromised keys immediately
- Apply **least privilege**: restrict keys to required scopes

```
1 import os
2 from dotenv import load_dotenv
3 load_dotenv() # loads from .env file
4 api_key = os.getenv("OPENAI_API_KEY")
```

Listing 12.3: Secure API Key Loading

12.4 Costs, Billing and Rate Limits

Pricing model: APIs are priced per **token** (not per request).

- **Input tokens:** prompt + context
- **Output tokens:** generated response
- 1 token $\approx$ 0.75 words; 1000 tokens $\approx$ 750 words

Rate limits are set per:

- **RPM** (Requests Per Minute)
- **TPM** (Tokens Per Minute)
- **TPD** (Tokens Per Day)

Best practices for cost control:

- Cache repeated requests

- Use smaller/cheaper models for simpler tasks
- Set `max_tokens` limits
- Batch requests where possible
- Monitor usage with dashboards; set spending alerts

12.5 Word Embeddings via API

```
1 response = client.embeddings.create(  
2     model="text-embedding-3-small",  
3     input="Machine learning is a subset of AI"  
4 )  
5 embedding = response.data[0].embedding # List of floats, dim  
    =1536
```

Listing 12.4: OpenAI Embeddings API

Use cases: Semantic search, clustering, classification, RAG retrieval, similarity scoring.

Similarity metric:

$$\text{cosine_similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

Chapter 13

Lecture 13: Prompt Engineering

13.1 Prompt Design Principles

The practice of designing and optimising input prompts to elicit desired, accurate, and useful outputs from LLMs without modifying model weights.

Core principles:

1. **Clarity:** Be specific and unambiguous about what you want
2. **Context:** Provide relevant background information
3. **Constraints:** Specify format, length, tone, and audience
4. **Examples:** Show the model what good output looks like
5. **Decomposition:** Break complex tasks into sub-tasks
6. **Persona:** Assign a role to the model to shape behaviour

13.2 Prompt Types

13.2.1 Zero-Shot Prompting

No examples provided. The model relies solely on its pre-trained knowledge.

```
1 Classify the sentiment of the following review as
2 Positive, Negative, or Neutral:
3 "The product broke after one day."
```

Listing 13.1: Zero-Shot

13.2.2 One-Shot Prompting

One example is provided before the task:

```
1 Text: "The service was excellent!" -> Sentiment: Positive
2 Text: "The product broke after one day." -> Sentiment:
```

Listing 13.2: One-Shot

13.2.3 Few-Shot Prompting

Multiple examples are provided. Strongly improves performance on complex tasks:

```
1 Text: "The service was excellent!" -> Sentiment: Positive
2 Text: "Completely useless product." -> Sentiment: Negative
3 Text: "It's okay, nothing special." -> Sentiment: Neutral
4 Text: "The product broke after one day." -> Sentiment:
```

Listing 13.3: Few-Shot

13.2.4 Chain-of-Thought (CoT) Prompting

Encourages the model to reason step-by-step before giving an answer:

```
1 Q: A store has 3 types of shirts at $15, $25, and $40.
2 You buy 2 of each. What is the total cost?
3
4 A: Let me think step by step.
5 - 2 shirts at $15 = $30
6 - 2 shirts at $25 = $50
7 - 2 shirts at $40 = $80
8 - Total = $30 + $50 + $80 = $160
9 The answer is $160.
```

Listing 13.4: Chain-of-Thought

Zero-Shot CoT: Adding “Let’s think step by step.” to the prompt significantly improves reasoning.

13.2.5 Role Prompting

Assign a persona to the model:

```
1 You are an expert cybersecurity analyst with 20 years
2 of experience. Explain SQL injection attacks to a
3 junior developer in simple terms.
```

Listing 13.5: Role Prompting

13.3 Evaluating LLM Responses

Criterion	Description
Correctness	Is the output factually accurate?
Relevance	Does it address the question asked?
Completeness	Does it cover all required aspects?
Conciseness	Is it appropriately brief without omissions?
Coherence	Is the output logically structured?
Faithfulness	Does it stay grounded in provided context?
Safety	Does it avoid harmful or biased content?

Table 13.1: LLM Response Evaluation Criteria

Automated evaluation: BLEU, ROUGE, BERTScore, LLM-as-judge.

13.4 Prompt Refinement Strategies

1. **Iterative refinement:** Start simple, identify failure modes, add constraints
2. **Negative instructions:** “Do NOT include...”
3. **Output format specification:** “Respond in JSON with keys: name, age, score”
4. **Self-consistency:** Sample multiple completions, take majority vote
5. **Decomposition:** “First do X. Then do Y.”
6. **Reflection:** Ask model to verify/critique its own output

Chapter 14

Lecture 14: Fine-Tuning

14.1 Fine-Tuning vs Prompting

Aspect	Prompting	Fine-Tuning
Model weights	Unchanged	Updated
Data required	Few to no examples	Hundreds to thousands
Cost	Low (only inference)	High (training compute)
Latency	Higher (long prompts)	Lower (shorter prompts)
Customisation	Limited	Deep specialisation
Consistency	Variable	Highly consistent

Table 14.1: Prompting vs Fine-Tuning

14.2 Situations for Fine-Tuning

Fine-tune when:

1. **Consistent style/format** is required (e.g., customer service tone)
2. **Domain-specific knowledge** is critical (e.g., medical, legal, finance)
3. **Prompt engineering fails** to achieve required quality
4. **Latency/cost** must be minimised (shorter prompts needed)
5. **Proprietary/confidential data** cannot be included in every prompt
6. **Safety and alignment** customisation is required

14.3 Instruction-Tuning

Fine-tuning a pre-trained LLM on a dataset of (instruction, output) pairs to make it follow natural language instructions more reliably.

Dataset format: Instruction-following examples:

```

1  ### Instruction:
2  Summarise the following article in 3 bullet points.
3
4  ### Input:
5  [Article text here]
6
7  ### Response:
8  - Point 1
9  - Point 2
10 - Point 3

```

Listing 14.1: FLAN/Alpaca-style instruction format

RLHF (Reinforcement Learning from Human Feedback):

1. **SFT:** Supervised fine-tuning on human demonstrations
2. **Reward model:** Train on human preference rankings
3. **RL:** Use PPO (Proximal Policy Optimisation) to optimise model against reward model

Parameter-Efficient Fine-Tuning (PEFT):

- **LoRA (Low-Rank Adaptation):** Freezes pre-trained weights; adds trainable low-rank decomposition matrices $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$
- **QLoRA:** LoRA with 4-bit quantised base model; enables fine-tuning on consumer GPUs
- **Prefix Tuning, Prompt Tuning, Adapter layers**

14.4 Knowledge Sourcing

Pre-training data: Web crawl (Common Crawl), books (Books1/2/3), Wikipedia, code (GitHub), academic papers.

Fine-tuning data sources:

- Human-curated annotation (expensive, high quality)
- Synthetic data generation (GPT-4 generates examples)
- Distillation from larger models (student-teacher)
- Domain-specific datasets (PubMed for biomedical, SEC filings for finance)

Chapter 15

Lecture 15: Managing State in Chatbots

15.1 Memory Challenges in Chatbots

LLMs are **stateless** by default – each API call is independent. Chatbots must explicitly manage conversation state.

Key challenges:

- **Context window limit:** Finite maximum tokens (e.g., 128K for GPT-4o). Long conversations exceed this.
- **Recency bias:** Models attend more to recent context
- **Cost:** Sending full history every turn is expensive
- **Relevance:** Not all history is relevant to current query

15.2 Multi-Turn Conversation Strategies

15.2.1 Full History

Send complete message history every turn. Simple but hits token limits.

15.2.2 Windowed History

Keep only the last k turns:

```
1 def get_windowed_messages(history, k=10):  
2     return history[-k:]    # last k exchanges
```

Listing 15.1: Windowed History

15.2.3 Summary Memory

Periodically compress old history into a summary:

```
1 def summarize_history(messages):
2     summary_prompt = f"Summarize this conversation:\n{messages}"
3     return llm.generate(summary_prompt)
4
5 # Use summary + recent turns
6 context = [{"role": "system", "content": f"Summary: {summary}"}]
    + recent_turns
```

Listing 15.2: Summary Memory

15.3 Context Management

Types of memory:

- **Short-term (buffer) memory:** Recent conversation turns
- **Long-term memory:** Vector database storing past interactions; retrieved by semantic similarity
- **Entity memory:** Track specific entities (people, facts) mentioned in conversation
- **Episodic memory:** Store and retrieve past conversation episodes

Token budget management:

Available tokens = Context window – System prompt – Response budget

15.4 Passing Conversation History

The **standard format** for the OpenAI API:

```
1 messages = [
2     {"role": "system", "content": "You are a helpful tutor."},
3     {"role": "user", "content": "What is gradient descent?"},
4     {"role": "assistant", "content": "Gradient descent is an
5         optimisation..."},
6     {"role": "user", "content": "Can you give an example?"},
7 ]
8 response = client.chat.completions.create(
9     model="gpt-4o", messages=messages
10 )
```

Listing 15.3: Conversation History Format

Roles:

- **system:** Persistent instructions and persona
- **user:** Human messages
- **assistant:** Model responses (included in subsequent calls)

Chapter 16

Lecture 16: Retrieval-Augmented Generation (RAG)

16.1 RAG Overview and Beyond Model Knowledge

RAG combines **retrieval** from an external knowledge base with **generation** by an LLM. Instead of relying solely on parametric knowledge (weights), the model is provided with relevant retrieved documents as context.

Motivation:

- LLMs have a **knowledge cutoff** – they don't know recent events
- LLMs **hallucinate** facts not in training data
- **Private/enterprise data** was never in the training corpus
- RAG provides **citations** and traceability

RAG Pipeline:

1. **Indexing (offline)**: Split documents into chunks → embed each chunk → store in vector DB
2. **Retrieval (online)**: Embed user query → find top- k similar chunks (by cosine similarity)
3. **Generation**: Pass retrieved chunks + user query to LLM → generate grounded response

16.2 Grounding LLMs with External Data Integration

Chunking strategies:

- **Fixed-size**: Split every n tokens with overlap (e.g., 512 tokens, 50-token overlap)

- **Sentence/paragraph-based:** Respect natural boundaries
- **Semantic chunking:** Use embedding similarity to detect topic boundaries
- **Hierarchical:** Parent-child chunk relationships

Retrieval methods:

- **Sparse retrieval (BM25):** Keyword-based; fast, no GPU needed
- **Dense retrieval:** Embedding-based; captures semantic similarity
- **Hybrid retrieval:** Combine BM25 + dense; often best in practice

16.3 Vector Databases

16.3.1 Chroma

Open-source, in-process vector database. Ideal for prototyping:

```

1 import chromadb
2 client = chromadb.Client()
3 collection = client.create_collection("my_docs")
4
5 # Add documents
6 collection.add(
7     documents=["ML is about learning from data"],
8     embeddings=[[0.1, 0.2, 0.3, ...]], # or auto-embed
9     ids=["doc1"]
10 )
11
12 # Query
13 results = collection.query(query_texts=["What is ML?"],
14                             n_results=3)

```

Listing 16.1: Chroma Vector DB

16.3.2 FAISS (Facebook AI Similarity Search)

High-performance similarity search library by Meta. Designed for billion-scale datasets:

```

1 import faiss
2 import numpy as np
3
4 d = 1536 # embedding dimension
5 index = faiss.IndexFlatL2(d) # L2 distance
6 index.add(embeddings_matrix) # numpy array, shape (N, d)
7 D, I = index.search(query_embedding, k=5) # top-5 results

```

Listing 16.2: FAISS

Index types:

- **IndexFlatL2:** Exact search, brute force

- **IndexIVFFlat**: Inverted file index; approximate, much faster
- **IndexHNSW**: Hierarchical Navigable Small World; graph-based ANN

16.4 Embedding Models

Model	Provider	Dim	Context
text-embedding-3-small	OpenAI	1536	8191 tokens
text-embedding-3-large	OpenAI	3072	8191 tokens
all-MiniLM-L6-v2	Sentence-Transformers	384	256 tokens
bge-large-en-v1.5	BAAI	1024	512 tokens
e5-large-v2	Microsoft	1024	512 tokens

Table 16.1: Popular Embedding Models

Chapter 17

Lecture 17: Rapid Prototyping Tools

17.1 GenAI Rapid Prototyping

Goal: Quickly build interactive demos and proof-of-concept applications without extensive frontend engineering.

17.2 HTML/CSS Limitations

For ML/GenAI prototyping, raw HTML/CSS has several drawbacks:

- No built-in ML/AI component library
- Requires JavaScript for interactivity
- Manual API integration
- No automatic reactivity – must handle state manually
- Long development cycle for data scientists not trained in web dev

17.3 Introduction to Streamlit

An open-source Python library that turns Python scripts into interactive web applications with minimal code. Automatically reruns script on user interaction.

Install: `pip install streamlit`

Run: `streamlit run app.py`

```
1 import streamlit as st
2 from openai import OpenAI
3
4 st.title("AI Chatbot")
5 client = OpenAI(api_key=st.secrets["OPENAI_API_KEY"])
6
7 if "messages" not in st.session_state:
```

```

8     st.session_state.messages = []
9
10    # Display chat history
11    for msg in st.session_state.messages:
12        st.chat_message(msg["role"]).write(msg["content"])
13
14    # Input
15    if prompt := st.chat_input("Ask me anything..."):
16        st.session_state.messages.append({"role": "user", "content":
17                                         : prompt})
18        st.chat_message("user").write(prompt)
19
20        response = client.chat.completions.create(
21            model="gpt-4o",
22            messages=st.session_state.messages
23        )
24        answer = response.choices[0].message.content
25        st.session_state.messages.append({"role": "assistant", "content": answer})
26        st.chat_message("assistant").write(answer)

```

Listing 17.1: Streamlit GenAI App Example

Key Streamlit features:

- **Widgets:** `st.text_input`, `st.slider`, `st.selectbox`, `st.file_uploader`
- **Display:** `st.write`, `st.dataframe`, `st.plotly_chart`, `st.image`
- **State:** `st.session_state` for persistent variables across reruns
- **Layout:** `st.columns`, `st.sidebar`, `st.expander`
- **Caching:** `@st.cache_data`, `@st.cache_resource`

17.4 Gradio Introduction

An open-source Python library for building ML model demos with a focus on ease of sharing via public links. Integrates deeply with Hugging Face Hub.

Install: `pip install gradio`

```

1    import gradio as gr
2    from openai import OpenAI
3
4    client = OpenAI()
5
6    def chat(message, history):
7        messages = [{"role": "user", "content": m}
8                    for m in history] + [{"role": "user", "content":
9                                         : message}]

```

```

9     response = client.chat.completions.create(
10         model="gpt-4o", messages=messages
11     )
12     return response.choices[0].message.content
13
14 demo = gr.ChatInterface(fn=chat, title="My GenAI Chatbot")
15 demo.launch(share=True)    # share=True creates a public URL

```

Listing 17.2: Gradio Chat Interface

Gradio vs Streamlit:

Feature	Streamlit	Gradio
Learning curve	Low	Very low
Flexibility	High	Moderate
HF Hub integration	Moderate	Native
Public sharing	Via cloud	One-click (<code>share=True</code>)
Best for	Full apps/dashboards	Model demos

Table 17.1: Streamlit vs Gradio Comparison

17.5 Building ML UIs with Python

Workflow for a RAG application:

1. **Upload:** `st.file_uploader` to accept PDF/TXT files
2. **Process:** Chunk, embed, store in Chroma/FAISS
3. **Query:** User inputs question, retrieve top- k chunks
4. **Generate:** Pass chunks + question to LLM
5. **Display:** Show answer with source citations

Chapter 18

Lecture 18: Agentic AI — Components

18.1 Key Components

An autonomous or semi-autonomous system that uses an LLM as its core reasoning engine to **perceive its environment**, **plan a sequence of actions**, and **execute tools** to achieve a goal.

18.1.1 LLM as Brain

The LLM:

- Interprets the task/goal
- Reasons about what to do next
- Selects which tool to call and with what arguments
- Synthesises tool results into a coherent response

18.1.2 Tools Integration

Tools extend the LLM's capabilities beyond text generation:

Tool Type	Examples
Web search	Bing, Google Search API, SerpAPI
Code execution	Python REPL, Jupyter kernel
API calls	Weather, Maps, Stock prices, Databases
File operations	Read/write files, parse PDFs
Database queries	SQL execution
Computer use	Browser, mouse/keyboard control

Table 18.1: Agent Tool Types

Function calling (OpenAI):

```
1 tools = [{
2     "type": "function",
3     "function": {
4         "name": "get_weather",
5         "description": "Get weather for a city",
6         "parameters": {
7             "type": "object",
8             "properties": {
9                 "city": {"type": "string"},
10                "unit": {"type": "string", "enum": ["celsius",
11                    "fahrenheit"]}
12            },
13            "required": ["city"]
14        }
15    }]
16 response = client.chat.completions.create(
17     model="gpt-4o", messages=messages, tools=tools
18 )
```

Listing 18.1: Tool/Function Calling

18.1.3 Memory Management

- **In-context (working) memory:** Current conversation + tool outputs in context window
- **External memory:** Vector DB for long-term facts and past interactions
- **State store:** Key-value store for structured agent state

18.2 Planning Mechanisms in Agentic AI

Task decomposition: Break complex goals into sub-tasks.

Planning strategies:

- **Chain-of-Thought:** Linear step-by-step reasoning
- **Tree of Thoughts (ToT):** Explore multiple reasoning branches
- **ReAct:** Reason then Act (interleaved) – see Lecture 19
- **MCTS:** Monte Carlo Tree Search for complex planning

18.3 Closed Source LLMs

18.3.1 GPT (OpenAI)

- GPT-4o: Multimodal, fast, most capable

- o1 / o3: Extended reasoning (“thinking”) models
- API access only (no open weights)

18.3.2 Gemini (Google)

- Gemini 2.0 Flash: Fast, cost-effective
- Gemini 2.0 Ultra: Most capable, multimodal
- Long context (2M tokens)

18.4 Open Source LLMs

18.4.1 Ollama

Tool for running LLMs locally:

```
1 ollama pull llama3.2      # download model
2 ollama run llama3.2      # interactive chat
3 ollama serve              # start REST API (port 11434)
```

Listing 18.2: Ollama Usage

```
1 import ollama
2 response = ollama.chat(
3     model='llama3.2',
4     messages=[{'role': 'user', 'content': 'Why is the sky blue?'
5             '}]
6 )
7 print(response['message']['content'])
```

Listing 18.3: Ollama Python API

18.4.2 LLaMA (Meta)

Open-weight family from Meta:

- LLaMA 3.2: 1B, 3B, 11B, 90B parameter models
- Available for commercial use (with Meta licence)
- Fine-tunable on consumer hardware with QLoRA

18.4.3 Gemma (Google)

Lightweight open models from Google:

- Gemma 2: 2B, 9B, 27B parameters
- Excellent performance-to-size ratio
- Available on Hugging Face, Ollama

Chapter 19

Lecture 19: Agentic AI — Control Flow

19.1 ReAct Framework: Reason + Act

A prompting framework that interleaves **Reasoning** traces with **Action** steps, allowing LLMs to reason about what to do, take actions (tool calls), observe results, and continue reasoning.

ReAct loop:

$\text{Thought}_t \rightarrow \text{Action}_t \rightarrow \text{Observation}_t \rightarrow \text{Thought}_{t+1} \rightarrow \dots \rightarrow \text{Final Answer}$

```
1 Question: What is the population of Tokyo?
2
3 Thought 1: I need to look up the current population of Tokyo.
4 Action 1: Search["Tokyo population 2024"]
5 Observation 1: Tokyo's population is approximately 13.96
   million.
6
7 Thought 2: I now have the answer.
8 Final Answer: Tokyo's population is approximately 13.96 million
   .
```

Listing 19.1: ReAct-style trace

Benefits:

- Interleaving reasoning with actions enables adaptive tool use
- Reasoning traces are interpretable
- Better at multi-step tasks than plain tool calling

19.2 State Management of AI Agents

Agent state includes:

- Current goal / task description
- Conversation history
- Available tools
- Results from past tool calls
- Scratch pad (working memory)
- Status: running, waiting for input, done

State update cycle:

$$s_{t+1} = f(s_t, a_t, o_t)$$

where s_t = state, a_t = action, o_t = observation.

19.3 Complex Control Flow

Patterns in agentic systems:

- **Sequential:** Agent executes steps in order
- **Conditional branching:** Different paths based on LLM decision or tool output
- **Looping:** Retry or refine until condition met (e.g., code passes tests)
- **Parallel:** Multiple agents working simultaneously
- **Human-in-the-loop:** Pause for human approval at critical steps
- **Multi-agent:** Orchestrator delegates sub-tasks to specialised agents

19.4 LangGraph

A library from LangChain for building **stateful, multi-actor applications** using a graph-based control flow. Nodes are functions/agents; edges define transitions.

19.4.1 Core Concepts

State: A shared TypedDict that flows through the graph:

```

1 from typing import TypedDict, Annotated
2 from langgraph.graph import StateGraph, END
3 import operator
4
5 class AgentState(TypedDict):
6     messages: Annotated[list, operator.add] # append-only
7     next_step: str

```

Listing 19.2: LangGraph State Definition

Nodes: Functions that take state and return updated state:

```
1 def agent_node(state: AgentState) -> AgentState:
2     # LLM decides what to do
3     response = llm.invoke(state["messages"])
4     return {"messages": [response]}
5
6 def tool_node(state: AgentState) -> AgentState:
7     # Execute tool calls from last message
8     results = tool_executor.batch(state["messages"][-1].
9         tool_calls)
10    return {"messages": results}
```

Listing 19.3: LangGraph Nodes

Edges: Define transitions between nodes:

```
1 builder = StateGraph(AgentState)
2 builder.add_node("agent", agent_node)
3 builder.add_node("tools", tool_node)
4
5 builder.set_entry_point("agent")
6
7 # Conditional edge: route based on LLM output
8 builder.add_conditional_edges(
9     "agent",
10    lambda s: "tools" if s["messages"][-1].tool_calls else END
11 )
12 builder.add_edge("tools", "agent") # after tools, back to
13    agent
14 graph = builder.compile()
```

Listing 19.4: LangGraph Edges and Compilation

Subgraphs: Compose graphs hierarchically for complex multi-agent systems.

Chapter 20

Lecture 20: Low-Code Automation

20.1 n8n Introduction and Visual Workflows

An open-source, self-hostable **workflow automation platform** with a visual node-based interface. Allows connecting apps, APIs, and AI services without writing full code. Pronounced “n-eight-n”.

Key features:

- Visual drag-and-drop workflow builder
- 400+ built-in integrations (Slack, Gmail, Notion, OpenAI, etc.)
- Self-hosted (data stays on your server) or cloud
- JavaScript expressions for data transformation
- Built-in AI nodes for LLM integration

Deployment options:

```
1 docker run -it --rm \  
2   -p 5678:5678 \  
3   -v ~/.n8n:/home/node/.n8n \  
4   n8nio/n8n  
5 # Access at http://localhost:5678
```

Listing 20.1: n8n Docker Setup

20.2 Nodes in n8n

Node types:

Node Type	Purpose
Trigger nodes	Start workflow (Webhook, Schedule, Manual, Event)
Action nodes	Perform operations (Send email, Create record)
Data transformation	Set, Merge, Split, Code (JavaScript)
Logic nodes	If/Else branching, Switch, Loop Over Items
AI nodes	OpenAI Chat, Embeddings, Summarise Chain
HTTP Request	Call any REST API

Table 20.1: n8n Node Types

Data model: Nodes pass **items** (JSON objects) to the next node. Each item can have multiple fields.

20.3 Managing Credentials in n8n

Credential types: API Key, OAuth2, Basic Auth, Bearer Token

Best practices:

- Store all secrets as n8n **Credentials** (encrypted at rest)
- Never hardcode keys in expressions
- Use **Environments** (n8n Cloud) for dev/staging/prod
- Credentials are referenced by name: `{{ $credentials.OpenAI.apiKey }}`
- Regularly audit and rotate credentials

20.4 Triggers in Agentic AI Systems

Types of triggers:

- **Webhook:** HTTP POST to a unique URL activates workflow (real-time event-driven)
- **Schedule (Cron):** Time-based trigger (every hour, daily at 9am)
- **Email:** New email received in Gmail/Outlook
- **Form submission:** n8n embedded form
- **Database change:** Polling or CDC (Change Data Capture)
- **Manual:** Test run in the editor
- **App event:** Slack message received, Notion page created, etc.

Example agentic trigger workflow:

1. User sends WhatsApp message (trigger)
2. Extract intent with OpenAI node
3. Conditional branch: booking vs inquiry vs complaint

4. For booking: query calendar, send confirmation email
5. For inquiry: RAG lookup from Notion database, reply
6. Log to Google Sheets

20.5 Replicating LangChain Logic in n8n

LangChain Concept	n8n Equivalent
LLMChain	OpenAI node with prompt template expressions
ConversationBufferMemory	Pass <code>sessionId</code> to OpenAI Chat node
VectorStore retriever	Pinecone / Supabase / Qdrant vector store nodes
RAG chain	Retrieve (vector store) + Set node + OpenAI node
Conditional router	If/Switch node
Tool calling / Agents	AI Agent node (built-in ReAct loop)
Output parser	Code node (JavaScript JSON parse)

Table 20.2: LangChain to n8n Mapping

n8n AI Agent node: The n8n **AI Agent** node implements the full ReAct loop internally:

1. Connect tools (Calculator, HTTP Request, Wikipedia, etc.)
2. Set system prompt and memory type
3. Node handles think-act-observe loop automatically
4. Returns final answer

n8n is excellent for integration-heavy workflows with existing tools. For complex custom logic, fine-grained control, or novel architectures, use LangChain/LangGraph directly in Python. Many production systems use n8n for orchestration and call Python services (FastAPI) for ML-intensive nodes.

Appendix A

Quick Reference: Key Formulae

Formula	Description
$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$	OLS normal equation
$\sigma(z) = \frac{1}{1+e^{-z}}$	Sigmoid function
$J = -\frac{1}{N} \sum [y \log \hat{p} + (1-y) \log(1-\hat{p})]$	Binary cross-entropy
$F_1 = \frac{2 \cdot P \cdot R}{P+R}$	F1 score
$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$	R-squared
$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$	Scaled dot-product attention
$\mathbf{h}_t = \phi(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b})$	RNN hidden state
$IG(S, A) = H(S) - \sum_v \frac{ S_v }{ S } H(S_v)$	Information gain
$\text{Gini}(S) = 1 - \sum_k p_k^2$	Gini impurity
$\text{RMSE} = \sqrt{\frac{1}{N} \sum (y_i - \hat{y}_i)^2}$	RMSE

Table A.1: Key Formulae Summary

Appendix B

Glossary

ANN Artificial Neural Network

API Application Programming Interface

BPE Byte-Pair Encoding – subword tokenisation algorithm

CART Classification and Regression Trees

CoT Chain-of-Thought prompting

DL Deep Learning

FFN Feed-Forward Network in Transformer

GAN Generative Adversarial Network

GELU Gaussian Error Linear Unit activation

GenAI Generative Artificial Intelligence

GPU Graphics Processing Unit

RLHF Reinforcement Learning from Human Feedback

KNN K-Nearest Neighbours

LLM Large Language Model

LoRA Low-Rank Adaptation – PEFT technique

LSTM Long Short-Term Memory network

MLP Multi-Layer Perceptron

NLP Natural Language Processing

OHE One-Hot Encoding

OLS Ordinary Least Squares

OOV Out-Of-Vocabulary

PEFT Parameter-Efficient Fine-Tuning

POS Part-of-Speech (tagging)

QLoRA Quantised Low-Rank Adaptation
RAG Retrieval-Augmented Generation
ReAct Reason + Act prompting framework
ReLU Rectified Linear Unit activation
RL Reinforcement Learning
RNN Recurrent Neural Network
RPM Requests Per Minute
SFT Supervised Fine-Tuning
TPM Tokens Per Minute
VAE Variational Autoencoder